



**UNIVERSIDADE FEDERAL DE PERNAMBUCO**  
**CENTRO DE INFORMÁTICA**

**PROJETO PARA A DISCIPLINA DE**  
**ESTRUTURA DE DADOS**  
**ORIENTADA A OBJETOS**

Alex Lacava  
Bruno Silva  
José Romildo  
Maria Cláudia  
Wallyson Silva

Recife, 2026

## **SUMÁRIO**

**I. Título do Projeto**

**II. Membros da Equipe**

**III. Organização e divisão da equipe**

**IV. Arquitetura**

**V. Composição para galeria de projetos**

**VI. Funcionalidades**

**VII. Conceitos utilizados**

**VIII. Link do vídeo**

## I. Título do Projeto

O **CIn-Events** permite que administradores gerenciem o ciclo de vida de eventos (Palestras, Workshops e Hackathons) e que alunos visualizem oportunidades de forma integrada. Este projeto é um sistema de gestão de eventos acadêmicos desenvolvido para a disciplina de Estrutura de Dados Orientada a Objetos no Centro de Informática da UFPE. O sistema utiliza uma arquitetura Full-stack, integrando um backend robusto em C++ com uma interface Single Page Application (SPA) em React. O objetivo do projeto foi treinar os conceitos do paradigma de Programação Orientada a Objeto aprendidos em sala.

## II. Membros da Equipe

1. Alex Lacava
2. Bruno Silva
3. José Romildo
4. Maria Cláudia
5. Wallyson Silva

## III. Organização e divisão da equipe

| Membro         | Responsabilidade                                                |
|----------------|-----------------------------------------------------------------|
| Alex Lacava    | Product Owner, Desenvolvedor Back-End                           |
| Bruno Silva    | Desenvolvedor Front-End, Desenvolvedor Back-End                 |
| José Romildo   | Desenvolvedor Back-End, Desenvolvedor Front-End, Documentação   |
| Maria Cláudia  | Desenvolvedora Back-End, Desenvolvedora Front-End, Documentação |
| Wallyson Silva | Desenvolvedor Back-End, Administrador do Banco de Dados         |

## IV. Arquitetura

```
C:.\
├── backend
│   ├── data           # Banco de dados/arquivos persistentes
│   ├── include        # Bibliotecas e headers C++
│   │   └── asio       # Biblioteca de comunicação HTTP/socket
│   └── src             # Código-fonte do backend
└── frontend
    └── src              # Código-fonte React
```

## V. Composição para galeria de projetos

**Figura 1.:** Tela inicial

> EVENTOS

> LOGIN

// CIn-Events\_

> identificar\_usuario.sh

LOG\_IN

CADASTRAR\_NOVO

**Figura 2.:** Tela cadastro

> EVENTOS

> LOGIN

// CIn-Events\_

> novo\_user.conf

Chico

francisco@cin.ufpe.br

Sistemas de Informação

\*\*\*\*\*

FINALIZAR\_CADASTRO

voltar

**Figura 3.:** Eventos (administrador)

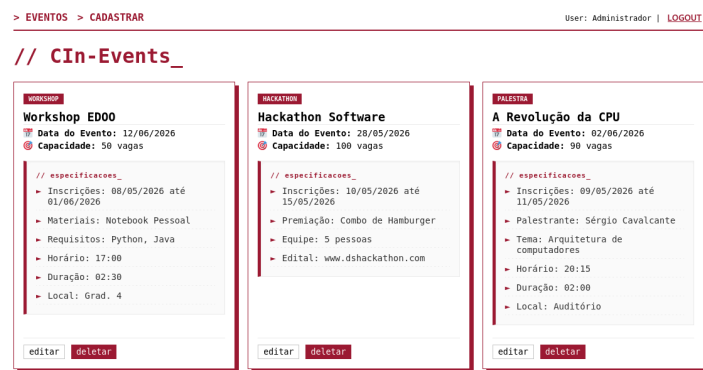


Figura 4.: Eventos (estudante)

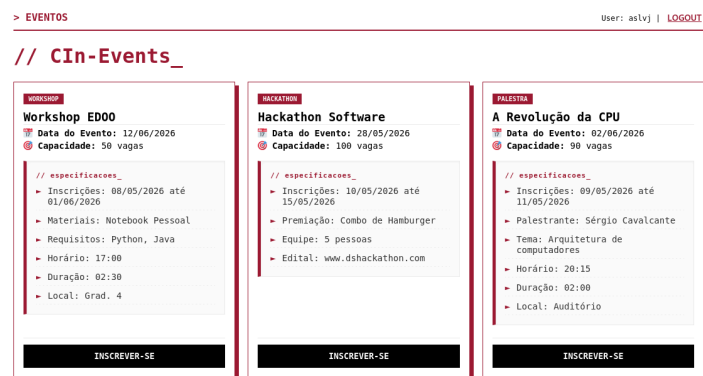


Figura 5. Cadastro de atividades

EVENTOS

CADASTRAR

User: Administrador | LOGOUT

// CIn-Events\_

> novo\_cadastro.sh

TIPO DE EVENTO:

Workshop

Título do Evento \*

Data (dd/mm/aaaa) \*

Início Inscrições \*

Fim Inscrições \*

Horário (hh:mm) \*

Local \*

Duração (hh:mm) \*

Limite de Vagas \*

DETALHES ESPECÍFICOS:

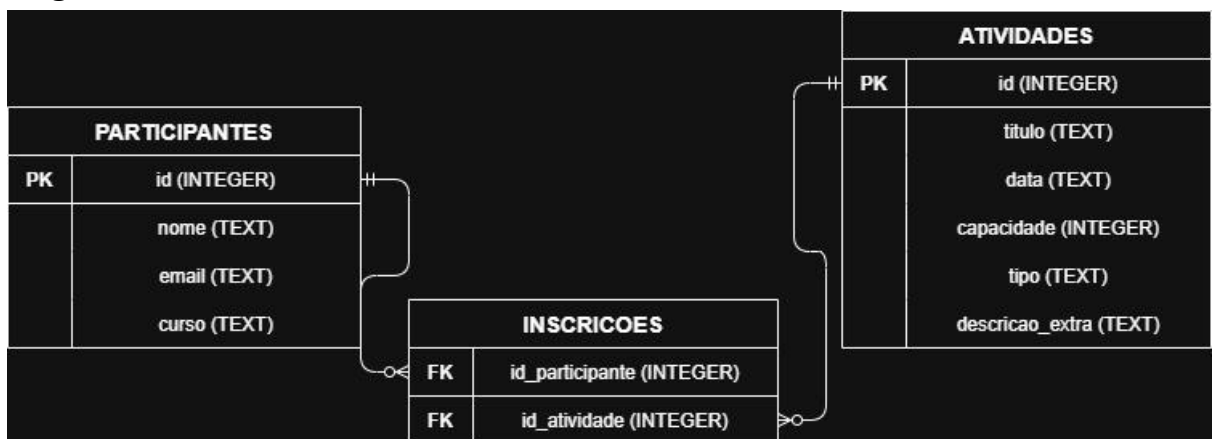
Materiais necessários

Software/Requisitos

EXEC\_CADASTRO

## VI. Funcionalidades

- **Arquitetura Client-Server:** Comunicação via API REST utilizando o framework Crow C++.
- **Persistência de Dados:** Integração com SQLite para armazenamento de atividades e JSON para persistência de usuários.
- **Sistema de Autenticação:**
  - Login flexível para usuários (aceita E-mail ou Nome).
  - Modo Administrador: Acesso restrito para criação, edição completa de todos os campos e exclusão de eventos.
- **Interface Web Moderna:**
  - Desenvolvida em React com roteamento dinâmico via react-router-dom.
  - Estética inspirada em terminais (Fonte JetBrains Mono) e identidade visual do CIn/UFPE.
  - Sistema de Notificações: Pop-ups customizados (Modais e Toasts) para feedback de ações do sistema (CRUD e Auth).
- **Gestão Dinâmica (CRUD):** Edição em tempo real de atributos como título, data, vagas, horário e local diretamente nos cards da interface.
- **Diagrama de Entidade e Relacionamento do Banco de Dados:**



## VII. Conceitos utilizados

O projeto aplica os pilares fundamentais de POO em um cenário real:

1. **Abstração:** Uso da classe abstrata Atividade como interface comum para todos os eventos.
2. **Herança:** Especialização das subclasses Palestra, Workshop e Hackathon.
3. **Polimorfismo Dinâmico:** Sobrescrita de métodos virtuais como getDescricaoExtra() e getTipo(), permitindo o processamento genérico de eventos via ponteiros da classe base.
4. **Encapsulamento:** Proteção de dados através de modificadores de acesso e gerenciamento de estado no React.

## VIII. Link do vídeo

[aqui](#)